

Dokumentacja techniczna projektu System zarządzania domową biblioteką

Sebastian Świąś

czerwiec 2026

Spis treści

1	Cel i zakres projektu	3
2	Technologie	3
3	Struktura katalogów	3
4	Konfiguracja budowania	4
5	Ogólna architektura	4
6	Model danych	4
6.1	Autor	4
6.2	Kategoria	5
6.3	Pozycja biblioteczna	5
7	Klasa Library	5
8	Obsługa plików CSV	6
9	Interfejs konsolowy	6
10	Główne scenariusze działania	7
10.1	Start programu	7
10.2	Dodanie pozycji	7
10.3	Filtrowanie po statusie	7
10.4	Zakończenie programu	7

11 Walidacja i obsługa błędów	8
12 Ograniczenia projektu	8
13 Podsumowanie	8

1 Cel i zakres projektu

Projekt jest konsolową aplikacją w języku C++, która służy do zarządzania domową biblioteką. Program pozwala przechowywać informacje o autorach, kategoriach oraz pozycjach bibliotecznych, na przykład książkach. Użytkownik może dodawać, edytować, usuwać i przeglądać dane, a także wyszukiwać pozycje i sprawdzać proste statystyki.

Aplikacja nie korzysta z bazy danych ani interfejsu graficznego. Dane są zapisywane w plikach CSV w katalogu `data`. Dzięki temu projekt pozostaje prosty, a format danych można łatwo podejrzeć w zwykłym edytorze tekstu.

Dokumentacja opisuje strukturę katalogów, najważniejsze klasy, sposób przechowywania danych, scenariusze działania programu oraz sposób budowania i uruchamiania projektu.

2 Technologie

Technologia	Zastosowanie
C++17	Implementacja modeli danych, logiki biblioteki oraz menu konsolowego.
STL	Wykorzystanie kontenerów, czyli gotowych struktur danych z biblioteki standardowej, na przykład <code>std::vector</code> i <code>std::map</code> .
CMake	Konfiguracja budowania projektu i tworzenie pliku wykonywalnego <code>HomeLibrary</code> .
CSV	Prosty zapis danych autorów, kategorii i pozycji bibliotecznych.
PowerShell	Pomocniczy skrypt <code>run.ps1</code> , który ułatwia budowanie i uruchamianie aplikacji w VS Code.

3 Struktura katalogów

Najważniejsze elementy projektu są podzielone według odpowiedzialności:

```
projekt_zaaawansowany_cpp/  
|-- CMakeLists.txt  
|-- run.ps1  
|-- data/  
| |-- authors.csv  
| |-- categories.csv  
| '-- items.csv  
'-- src/  
    |-- main.cpp  
    |-- models/  
    | |-- Author.h / Author.cpp  
    | |-- Category.h / Category.cpp  
    | '-- Item.h / Item.cpp  
    '-- core/
```

```
| '-- Library.h / Library.cpp  
|-- utils/  
    '-- FileStorage.h / FileStorage.cpp
```

Katalog `models` zawiera klasy opisujące dane. Katalog `core` zawiera główną logikę aplikacji, a katalog `utils` odpowiada za zapis i odczyt plików. Plik `main.cpp` zawiera menu tekstowe oraz obsługę danych wpisywanych przez użytkownika.

4 Konfiguracja budowania

Projekt jest konfigurowany przez plik `CMakeLists.txt`. Wymagany jest standard C++17. Plik wykonywalny ma nazwę `HomeLibrary`. Do programu dołączane są wszystkie pliki źródłowe z katalogów `models`, `core` i `utils`.

Przykładowe polecenia budowania:

```
cmake -B build  
cmake --build build  
.\build\HomeLibrary.exe
```

W projekcie znajduje się też skrypt `run.ps1`. Jest przydatny szczególnie w VS Code, ponieważ próbuje użyć CMake, a jeśli to się nie uda, kompiluje projekt przez `g++`.

```
powershell -ExecutionPolicy Bypass -File .\run.ps1
```

5 Ogólna architektura

Program można podzielić na cztery warstwy:

- warstwa interfejsu konsolowego w pliku `main.cpp`,
- warstwa logiki biblioteki w klasie `Library`,
- warstwa modeli danych: `Author`, `Category`, `Item`,
- warstwa zapisu i odczytu danych w klasie `FileStorage`.

Menu konsolowe pobiera dane od użytkownika i wywołuje metody klasy `Library`. Klasa `Library` przechowuje obiekty w pamięci programu i wykonuje operacje na listach. Klasa `FileStorage` zapisuje aktualny stan do plików CSV oraz wczytuje dane przy starcie programu.

6 Model danych

6.1 Autor

Klasa `Author` przechowuje identyfikator, imię i nazwisko autora. Udostępnia metody dostępowe, metody zmiany pól oraz konwersję do formatu CSV.

```
id;firstName;lastName
```

Przykład:

```
3;Olga;Tokarczuk
```

6.2 Kategoria

Klasa `Category` opisuje kategorię pozycji bibliotecznej. Każda kategoria ma identyfikator oraz nazwę.

```
id;name
```

Przykład:

```
2;Fantasy
```

6.3 Pozycja biblioteczna

Klasa `Item` opisuje pojedynczą pozycję w bibliotece. Zawiera tytuł, odwołanie do autora i kategorii, ocenę, status oraz opcjonalny opis.

```
id;title;authorId;categoryId;rating;status;description
```

Przykład:

```
4;Solaris;5;3;9.300000;przeczytane;Klasyka polskiej fantastyki naukowej.
```

Relacja między pozycją, autorem i kategorią jest realizowana przez pola `authorId` oraz `categoryId`. Program nie używa mechanizmu kluczy obcych z bazy danych, ale podczas dodawania lub edycji pozycji sprawdza, czy wybrany autor i kategoria istnieją.

7 Klasa Library

Klasa `Library` jest głównym miejscem przechowywania danych w czasie działania programu. Wewnątrz ma trzy kolekcje:

```
std::vector<Author> authors;  
std::vector<Category> categories;  
std::vector<Item> items;
```

Klasa udostępnia operacje CRUD dla autorów, kategorii i pozycji. Identyfikatory nowych rekordów są wyznaczane na podstawie największego aktualnego ID, powiększonego o jeden. Przykładowo metoda `getNextAuthorId()` przegląda listę autorów i zwraca kolejną wolną wartość.

W klasie znajdują się też metody wyszukiwania:

- wyszukiwanie pozycji po fragmencie tytułu,
- pobieranie pozycji konkretnego autora,
- pobieranie pozycji z wybranej kategorii,
- filtrowanie pozycji po statusie.

Statystyki są liczone na podstawie aktualnej listy pozycji. Program wyznacza między innymi średnią ocenę wszystkich pozycji, liczbę pozycji w kategoriach oraz średnią ocenę dla konkretnej kategorii.

8 Obsługa plików CSV

Za zapis i odczyt danych odpowiada klasa `FileStorage`. Każdy typ danych ma oddzielny plik:

Plik	Zawartość
<code>authors.csv</code>	Lista autorów.
<code>categories.csv</code>	Lista kategorii.
<code>items.csv</code>	Lista pozycji bibliotecznych.

Przy uruchomieniu aplikacja próbuje wczytać dane z katalogu `data`. Przy wyjściu dane są automatycznie zapisywane. W menu dostępne są również ręczne opcje zapisu i wczytania.

W obecnej wersji programu zapis został zabezpieczony przed prostą sytuacją, w której pusty stan programu mógłby nadpisać istniejące pliki danych. Jest to przydatne, gdy program zostanie uruchomiony z innego katalogu roboczego lub gdy nie uda się poprawnie wczytać danych.

9 Interfejs konsolowy

Interfejs użytkownika znajduje się w pliku `main.cpp`. Jest zbudowany z głównego menu oraz trzech podmenu:

- zarządzanie autorami,
- zarządzanie kategoriami,
- zarządzanie pozycjami.

Dodatkowo w menu głównym znajdują się statystyki oraz ręczny zapis i odczyt danych. Program korzysta z prostych funkcji pomocniczych do wczytywania wartości liczbowych i tekstowych. Dzięki temu błędne dane wpisane przez użytkownika nie powinny kończyć programu, tylko powodować wyświetlenie komunikatu.

Przykładowo ocena pozycji musi być liczbą z zakresu od 0 do 10, a wybór statusu jest ograniczony do trzech opcji:

- do przeczytania,
- w trakcie,
- przeczytane.

10 Główne scenariusze działania

10.1 Start programu

1. Program ustawia obsługę znaków w konsoli Windows.
2. Tworzony jest obiekt klasy `Library`.
3. Aplikacja szuka katalogu `data`.
4. Dane z plików CSV są wczytywane do pamięci.
5. Wyświetlane jest menu główne.

10.2 Dodanie pozycji

1. Użytkownik wybiera opcję dodania pozycji.
2. Program sprawdza, czy istnieje przynajmniej jeden autor i jedna kategoria.
3. Użytkownik podaje tytuł, autora, kategorię, ocenę, status i opcjonalny opis.
4. Program waliduje wybrane ID autora i kategorii.
5. Nowy obiekt `Item` zostaje dodany do kolekcji w klasie `Library`.

10.3 Filtrowanie po statusie

1. Użytkownik wybiera opcję filtrowania po statusie.
2. Program wyświetla listę dostępnych statusów.
3. Klasa `Library` przechodzi po liście pozycji i wybiera te, których status zgadza się z wybraną wartością.
4. Wyniki są wypisywane w konsoli razem z autorem, kategorią i oceną.

10.4 Zakończenie programu

1. Użytkownik wybiera opcję wyjścia.
2. Program zapisuje dane do plików CSV.
3. Po udanym zapisie wyświetlany jest komunikat końcowy.

11 Walidacja i obsługa błędów

Program zawiera podstawową walidację danych wpisywanych przez użytkownika. Funkcje `readIntInRange()` oraz `readDoubleInRange()` wymuszają zakres liczb. Funkcja `readRequiredLine()` pilnuje, aby wymagane pola tekstowe nie były puste.

Przy edycji pozycji program sprawdza, czy wpisane ID autora lub kategorii jest poprawną liczbą oraz czy wskazany obiekt istnieje. Jeżeli dane są błędne, poprzednia wartość zostaje zachowana, a użytkownik dostaje komunikat.

Wczytywanie z CSV jest również zabezpieczone przed prostymi błędami formatu. Jeżeli wiersz nie ma wymaganej liczby pól albo konwersja liczby się nie uda, tworzony jest pusty obiekt, który nie jest dodawany do biblioteki.

12 Ograniczenia projektu

Projekt jest celowo prosty i konsolowy. Najważniejsze ograniczenia to:

- brak graficznego interfejsu użytkownika,
- brak prawdziwej relacyjnej bazy danych,
- prosty format CSV bez obsługi średników wewnątrz tekstu,
- brak rozbudowanych testów automatycznych,
- brak kont użytkowników i uprawnień.

Te ograniczenia wynikają głównie z zakresu projektu. Dla małej domowej biblioteki plikowy zapis CSV jest wystarczający i łatwy do zrozumienia.

13 Podsumowanie

System zarządzania domową biblioteką jest prostą aplikacją konsolową napisaną w C++17. Projekt pokazuje wykorzystanie klas, enkapsulacji, kontenerów STL, na przykład `std::vector`, podziału na moduły oraz zapisu danych do plików. Najważniejszą klasą logiki jest `Library`, a za trwałość danych odpowiada `FileStorage`. Program realizuje podstawowe operacje CRUD, wyszukiwanie, filtrowanie po statusie oraz statystyki, dzięki czemu spełnia założenia małej aplikacji do zarządzania prywatną kolekcją książek.